Project 9 | MCA Projects 2026 | Section A

HEXARCH Study Guide

Instruction and Data Memory Hierarchy, explained from zero.

> **One sentence version: separate paths for instructions and data let a processor do two memory jobs in the same clock tick. A shared single-port memory has to do them one after another.**

What is in this guide

1. The big idea in plain words
2. Words you need to know
3. The tiny processor and its three-instruction program
4. The two designs side by side
5. Cycle by cycle: why 4 versus 6
6. The files: what each one does
7. The one rule that creates the difference
8. Tests, results and how to run everything
9. Roles and what the evaluation needs
0. Questions you may be asked
1. The video: how to record it, scene by scene

Team: Akshat Chauhan, Ajeesh Amreet, Vaibhav, Hardik Maheshwari, Mitul Bhatia. Phase I deadline: 16 November 2026, 11:59 PM IST.

Every number in this guide comes from a real simulation run (Icarus Verilog 13.0).

1. The big idea in plain words

A processor keeps asking memory two different questions:

- **"What should I do next?"** The answer is an **instruction**.
- **"What value am I working on?"** The answer is **data**.

Think of a kitchen

A cook needs a **recipe** (instructions) and **ingredients** (data).

Harvard: two counters

One counter hands out recipe steps. A different counter hands out ingredients. The cook can **read the next recipe step while grabbing an ingredient**. Nobody waits.

Von Neumann-style shared: one counter

One window serves both. If the cook is collecting an ingredient, the next recipe step **has to wait** until the window is free.

What this project does

The brief calls this **von Neumann vs Harvard**. We built both memory designs in **Verilog** (a language for describing digital hardware), gave them the **same tiny processor, same program, same data, same total memory**, simulated them, and counted the clock cycles each needs.

The result

Same answer in both (the value 7 gets copied to a new place). Harvard finishes in **4 cycles**. Shared memory needs **6 cycles**, because two instruction fetches had to wait for the single port.

An honest limit, say this out loud if asked

This proves a point about *port contention* in a tiny model. It does **not** prove "Harvard is always faster". A program with no data accesses takes 4 cycles in *both* designs (we tested it).

Where it sits in the real world

Many small microcontrollers (PIC, AVR) use separate instruction and data memories. Big CPUs usually present one shared memory but use separate instruction and data *caches*. That is why it is called a "memory hierarchy" project. The optional lower-level shared memory from the brief is **deliberately left out**, which the brief allows.

2. Words you need to know

Word	Meaning in this project
Instruction	A one-byte command such as LOAD, STORE, HALT.
Data	The value being worked on. Here: the number 7.
Memory	A table of bytes you read by address. Instruction memory: 16 bytes. Data memory: 16 bytes.
Port	One "door" into a memory: one address in, one value out per cycle. A single-port memory can serve only one access per cycle.
Clock / cycle	The heartbeat. Things change on the **rising edge**. In simulation one cycle is 10 ns (a test setting, not a measured hardware speed).
PC (program counter)	4-bit counter holding the address of the next instruction to fetch.
Fetch	Reading the next instruction from memory.
IR (instruction register)	Holds the instruction that is running *now* while the PC already points at the next one.
ACC (accumulator)	One 8-bit register that temporarily holds a value. LOAD fills it. STORE copies it into memory and ACC keeps its value.
Valid bit	`execute_valid` : "the IR holds a fresh instruction". Stops an old instruction from running twice when a fetch was blocked.
Harvard	Separate instruction and data memories with independent ports.
Von Neumann-style / shared single-port	One memory, one port, instructions and data take turns. Named after von Neumann's 1945 stored-program design.
Address decoder	In shared mode: picks the physical address. Instructions to 0-15, data to 16-31.
Stall / bubble	A cycle where the fetch is blocked, so nothing new enters the processor.
HALT	Instruction that stops the processor; it then makes no memory requests.
Testbench	Verilog code that runs the design and checks the answers automatically (`tb/tb_memory_compare.v`).
VCD / waveform	A recording of every signal over time that a viewer can plot.

3. The tiny processor and its program

Every instruction is one byte: two halves (nibbles)



Hex bytes in `programs/demo.hex` : **10 21 F0**

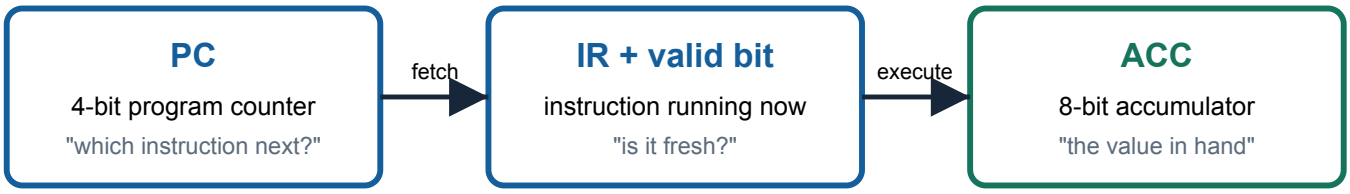
High nibble	Instruction	What it does
`1`	**LOAD a**	ACC ← data[a]
`2`	**STORE a**	data[a] ← ACC
`F`	**HALT**	Stop; no more memory requests
`0`	**NOP**	Do nothing
anything else	**illegal**	Stop and raise the `fault` flag

The demonstration program

```
LOAD 0     ; ACC = data[0]   (data[0] starts as 7)
STORE 1    ; data[1] = ACC   (now data[1] is 7 too)
HALT       ; stop
```

> It simply **copies the number 7 from data address 0 to data address 1**. Both designs must end with data[1] = 7 and ACC = 7.

The processor's three parts



"Fetch" really goes through memory: the PC supplies an address, the instruction memory returns the byte, and the byte is stored in the IR.

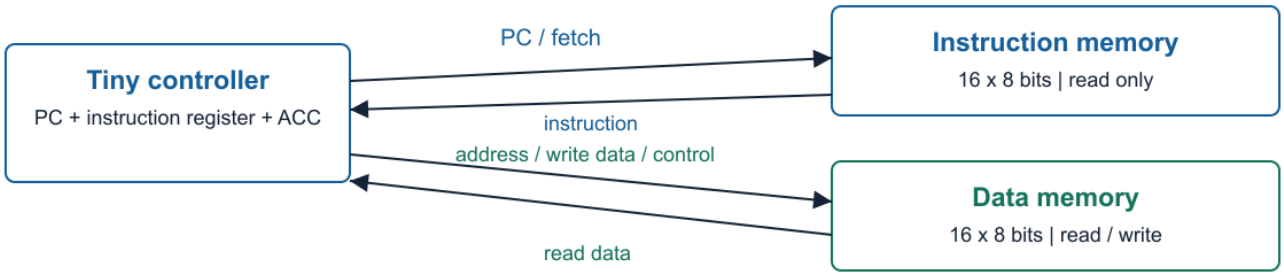
Having **one instruction register** is what lets the processor **execute the current instruction while fetching the next**. That overlap is the only pipelining in the design, and it is exactly what the Harvard memory makes possible.

4. The two designs side by side

One controller. Two memory organizations.

Both store 16 instruction bytes and 16 data bytes.

HARVARD: independent instruction and data ports



Fetch the next instruction while the current LOAD / STORE accesses data.

SHARED (VON NEUMANN-STYLE): one port must take turns



Addresses 0-15: instructions. Addresses 16-31: data. LOAD / STORE pauses fetch.

	Harvard	Shared single-port
Memory	16-byte instruction + 16-byte data	One 32-byte memory
Ports	Two independent	One, taken in turns
Addresses	Each memory starts at 0	0-15 instructions, 16-31 data (address decoder)
Fetch during LOAD/STORE?	Yes, same cycle	No: data wins, fetch is blocked, PC holds
Same for both	Controller, program, starting data, total capacity (32 bytes), clock period	

In the code, one parameter chooses the design: `SHARED_BUS = 0` is Harvard, `SHARED_BUS = 1` is shared (in `rtl/memory_system.v`). The same controller runs on both, so the only thing that differs is how the memory is wired.

5. Cycle by cycle: why 4 versus 6

Read each row left to right. Light box = **fetching** that instruction. Dark box = **executing** it.

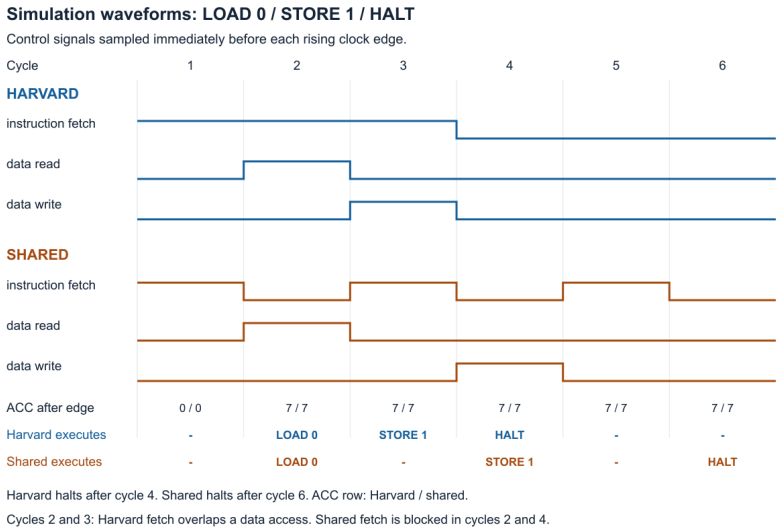
Harvard: 4 cycles

	1	2	3	4	5	6
Fetching	LOAD 0	STORE 1	HALT	none	stopped	stopped
Executing	empty	LOAD 0 reads 7	STORE 1 writes 7	HALT	stopped	stopped

Shared: 6 cycles

	1	2	3	4	5	6
Fetching	LOAD 0	BLOCKED	STORE 1	BLOCKED	HALT	none
Executing	empty	LOAD 0 reads 7	empty	STORE 1 writes 7	empty	HALT

> **Cycle 2 is the key moment.** Harvard's fetch and data read signals are both 1 at once. In shared memory the fetch signal is 0, because the only port is busy reading data. The same thing happens in cycle 4 when STORE writes. Two blocked fetches = two extra cycles = 6 instead of 4.

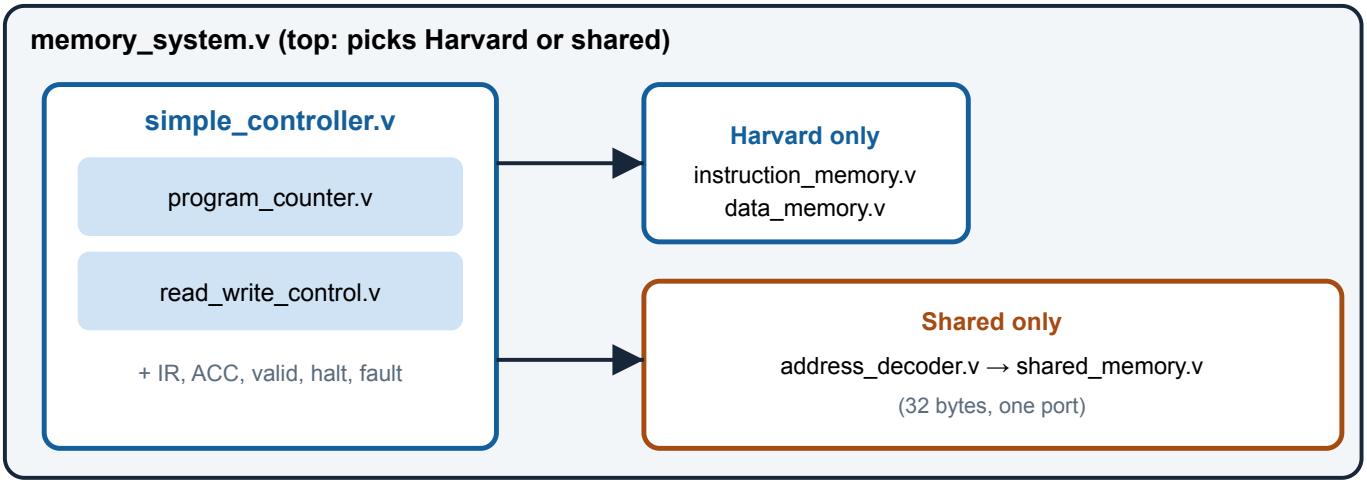


Same story as lines going up and down. Blue (Harvard): fetch stays high while read and write pulse. Orange (shared): fetch drops every time read or write is high.

Why "empty" in cycles 3 and 5 (shared)? A blocked fetch leaves no fresh instruction. The instruction register still holds the previous one (LOAD in cycle 3, STORE in cycle 5), but its valid bit is 0, so it is not executed again.

> **The general rule.** For a straight-line program of **N** instructions (including HALT) with **M** LOAD/STORE instructions: Harvard takes **N + 1** cycles, shared takes **N + 1 + M**. The demo has N = 3, M = 2, so 4 and 6. Every test and 2000 random programs follow it.

6. The files: what each one does



File (rtl/)	Job	Used in
`program_counter.v`	Counts up the instruction address, but **only when a fetch is accepted**.	Both
`instruction_memory.v`	16 bytes, loaded from `programs/demo.hex`. Read-only.	Harvard
`data_memory.v`	16 bytes, loaded from `programs/data.hex`. Read; write on the clock edge. Reset does not erase it.	Harvard
`read_write_control.v`	Decodes the instruction's top nibble into LOAD / STORE / HALT / illegal.	Both
`simple_controller.v`	The brain: holds IR, ACC, valid, halted, fault; decides when to fetch and when to read/write.	Both
`address_decoder.v`	One-line mux: data access gives address 16-31, otherwise 0-15.	Shared
`shared_memory.v`	32 bytes behind a single port; same read/write behaviour.	Shared
`memory_system.v`	Plugs the same controller into either memory arrangement.	Both

Other things in the repository

`tb/tb_memory_compare.v`	Runs both designs side by side, counts cycles, and **fails loudly** if anything is wrong.
`tb/tb_random_stress.v`	2000 random programs checked against a software model and the N + 1 / N + 1 + M rule.
`programs/*.hex`	The program (`10 21 F0 ...`) and the starting data (`07 00 00 ...`).
`results/`	Saved outputs: cycle table, CSVs, waveforms (`.vcd`).
`docs/` , `output/pdf/`	Explanations, requirement checklist, IEEE report (LaTeX source + PDF), figures, video script.
`Makefile`	Shortcuts: `make demo` , `make test` , `make stress` , `make figures` , `make report` , `make guide` .

7. The one rule that creates the difference

Open `rtl/simple_controller.v`. Everything above comes from these three lines:

```
assign data_read_enable  = running && execute_valid && load;
assign data_write_enable = running && execute_valid && store;
assign fetch_enable = running && !stop_now && !(SHARED_BUS && data_access);
```

- **LOAD / STORE** raise a data read / write signal while their instruction is running.
- **Fetch** is allowed unless the processor is halting, **or** (shared mode *and* a data access is happening).
- In Harvard mode `SHARED_BUS` is 0, so that last part is always false: fetch never waits.

What blocking a fetch does

1. `fetch_enable` = 0, so the PC does not advance (it only counts when a fetch is accepted).
2. The valid bit is loaded with 0, so next cycle nothing runs twice.
3. Next cycle the data access is over, `fetch_enable` = 1, and the delayed instruction is fetched.

Timing rules in one box

> 1. **Reads** are instant (combinational). **LOAD** captures the value at the rising edge. **STORE** writes at the rising edge.
> 2. A shared data access blocks the fetch, so the PC holds for one cycle.
> 3. **Reset** clears the controller and suppresses all requests, but keeps memory contents. **HALT** stops all further requests.

The address decoder (shared mode)

```
physical_address = data_select ? {1'b1, data_address}      // 16..31 : data
                               : {1'b0, instruction_address}; //  0..15 : instructions
```

The top bit of the 5-bit address says "instruction side" (0) or "data side" (1). That is the whole mapping.

Known limits of the model (good to know before an interview)

- Programs must fit in 16 bytes and finish with HALT. The 4-bit PC wraps around otherwise.
- No jumps or branches, no arithmetic. Only LOAD, STORE, HALT, NOP.
- Memory reads are combinational and writes happen on the rising edge: this is a simplified model, not a specific chip.
- The 10 ns clock is a simulation setting. FPGA resources and real frequency were **not** measured.

8. Tests, results and how to run everything

Run it yourself (needs Icarus Verilog, Make, Python 3)

```
git clone https://github.com/mitul-bhatia/HEXARCH.git
cd HEXARCH
make demo     # the 6-cycle table, prints DEMO PASSED
make test     # every scenario, prints ALL TESTS PASSED
make stress   # 2000 random programs, prints STRESS PASSED
```

Install on macOS: `brew install icarus-verilog`. Ubuntu/Debian: `sudo apt-get install iverilog make python3`. Windows: install WSL (`wsl --install -d Ubuntu`), then use the Ubuntu command inside Ubuntu.

The six scenarios and the results we measured

#	Scenario	Why it exists	Harvard	Shared
1	**demo**: LOAD 0, STORE 1, HALT	The main comparison	4	6
2	**address boundaries**	Uses data addresses 0 and 15; a STORE right before a LOAD of the same address	6	10
3	**no data access**: two NOPs, HALT	Shows the gain depends on the program	4	4
4	**HALT only**	Stops immediately, no traffic	2	2
5	**illegal opcode**	Stops with the fault flag set	2	2
6	**reset and restart**	Reset mid-program, memory kept, restart, longer copy chain	8	14

Cycle count starts at the first clock edge after reset is released and includes the HALT cycle.

What the testbench checks automatically

✓ Every data word ends with the expected value, in both designs.
✓ Instructions are never changed.
✓ Each LOAD / STORE happens exactly once (no repeats).
✓ PC advances only on a fetch, and holds during a blocked fetch.
✓ In shared mode, fetch and data access never happen in the same cycle.
✓ After HALT nothing touches memory; ACC is held.

Where the evidence lives

`results/demo_output.txt`	The readable cycle table from `make demo` (F/R/W = fetch / read / write).
`results/test_output.txt`, `summary.csv`	All scenario results.
`results/stress_output.txt`	Random-program result.
`results/demo.vcd`	Waveform. Open in a viewer such as GTKWave. Look under `tb_memory_compare.h` (Harvard) and `.s` (shared).
`output/pdf/HEXARCH_Phase_I_Report.pdf`	IEEE-format Phase I report.

9. Roles and what the evaluation needs

Who owns what

The same split is used in the report (Table II), the video and the starter tasks.

Member	Owns	Presents in the video
Akshat Chauhan	Instruction path: `program_counter.v`, `instruction_memory.v`	Scene 2: the idea, the program counter
Ajeesh Amreet	Data path: `data_memory.v`, `read_write_control.v`	Scene 3: the program, data-memory timing
Vaibhav	Shared memory: `shared_memory.v`, `address_decoder.v`, the fetch-stall rule	Scene 4: why shared memory waits
Hardik Maheshwari	Simulation: testbenches, results, waveforms	Scenes 5-6: `make demo`, waveform, `make test`
Mitul Bhatia	Integration: `memory_system.v`, `simple_controller.v`, build, report, repository, video	Scenes 1 and 7, and drives the recording

Phase I rubric (30%) and where each item is

Requirement	Marks	Where it is in the repository
IEEE report: abstract, literature review, proposed solution, individual contributions, references	10	`output/pdf/HEXARCH_Phase_I_Report.pdf` (source in `docs/report/`)
Video, at most 5 minutes: idea, all five names, working prototype	15	Recording guide: `docs/VIDEO_SCRIPT.md` and section 11 below. Link goes in `VIDEO.md`
GitHub: report, video link, prototype files, artefacts	5	The repository itself: `rtl/`, `tb/`, `results/`, `docs/`, `output/pdf/`
Every member's meaningful contribution in the commit history	(part of the 5)	Each member's own pull request: `docs/MEMBER_TASKS.md`

Project brief and where each item is

Brief item	Evidence
Von Neumann vs Harvard	Same controller, `SHARED_BUS` = 0 (Harvard) or 1 (von Neumann-style)
Program counter, instruction memory, data memory	`program_counter.v`, `instruction_memory.v`, `data_memory.v`
Address decoder, read/write control	`address_decoder.v`, `read_write_control.v` (enables gated in `simple_controller.v`)
Analyze simultaneous access opportunities	Cycle table, overlap and stall counts, the N + 1 / N + 1 + M rule
RTL, block diagram, waveforms, comparison	`rtl/`, `docs/figures/`, `results/*.vcd`, report Table I
Optional shared lower-level memory	Not built (optional); a Phase II extension

Deadline: 16 November 2026, 11:59 PM IST. The plan and checklist are in `docs/TEAM_AND_PLAN.md`.

10. Questions you may be asked

Question	Short answer
What is a Harvard architecture?	Separate memories and buses for instructions and data, so both can be accessed in the same cycle.
What is the shared alternative called?	Von Neumann style: one memory for both. With a single port, only one access per cycle.
Why 4 cycles versus 6?	Two of the program's instructions (LOAD, STORE) use data memory. In shared mode each blocks the next fetch for one cycle: 4 + 2 = 6.
Is Harvard always faster?	No. With no data accesses both take 4 cycles (our no-data test). It helps when fetch and data access would compete.
Why is there a valid bit?	When a fetch is blocked the IR still holds the old instruction. The valid bit says "nothing new", so it does not run twice.
Why only one instruction register?	That is enough to overlap "execute current" with "fetch next", which is all the demo needs.
What does reset do?	Clears PC, ACC, IR, valid, halted, fault and stops memory requests. Memory contents are kept (tested).
What happens with an unknown opcode?	The controller halts and sets the `fault` flag (tested).
What if a program has no HALT?	The 4-bit PC wraps after 16 instructions and the program repeats. The unused image bytes are filled with HALT (`F0`) to avoid this in our programs.
How do you know both designs are fair?	Same controller, same program and data, same 32-byte total, same clock period. Only the memory wiring differs.
How did you verify correctness?	An automatic testbench (six scenarios) and a 2000-program random test check every data word, access counts, PC behaviour, port ownership and halt behaviour.
What is not covered?	Caches, branches, a full instruction set, synthesis or FPGA measurement, real clock speed. The optional lower-level shared memory is omitted.
Did you use AI?	The repository states starter code and drafts were prepared with AI assistance. Be ready to explain your own part and your own commits honestly.
Is there a general formula?	Yes. Harvard = N + 1 cycles, shared = N + 1 + M, for N instructions with M LOAD/STORE. Checked on every test and 2000 random programs.
Why "hierarchy" if there is one level?	The project title is the course's. We implement the instruction/data split that sits at the top of real hierarchies (split L1 caches). The optional lower level is a Phase II extension.
What does F/R/W mean in the output?	Instruction fetch / data read / data write.

If they ask you to open a file and explain

- **Waveform:** open `results/demo.vcd`. Show `clk`, `pc`, `instruction_register`, `execute_valid`, `fetch_enable`, `data_read_enable`, `data_write_enable`, `accumulator`, `halted` for both `h` and `s`.
- **Code:** go straight to the `fetch_enable` line in `simple_controller.v`, then `address_decoder.v`.

11. The video: how to record it, scene by scene

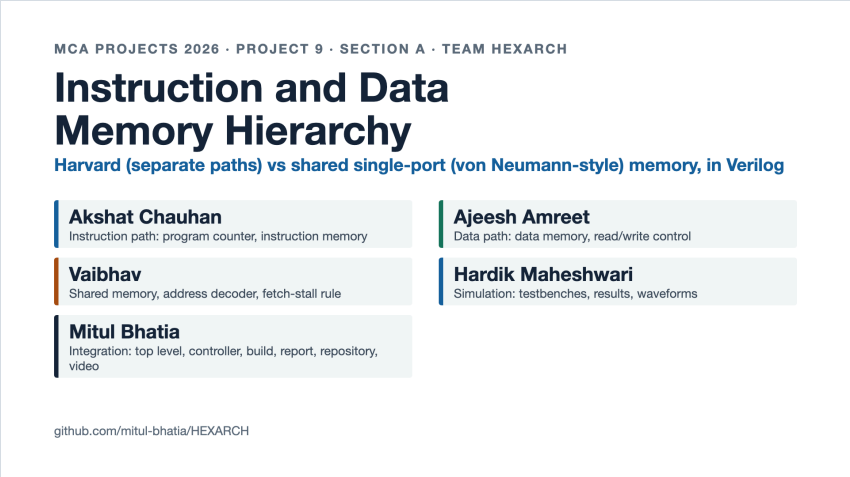
> **Must have:** at most 5:00 · the project idea · all five full names · the working prototype. **This plan:** 7 scenes, about 3:40 of speech, about 4:40 with screen actions.

How it works

- **One laptop, one driver.** Mitul's laptop shows everything. Mitul types the commands and switches windows when the speaker says the **[cue word]**. Speakers do not read the bracketed cue words aloud and never touch the keyboard.
- **Record:** in the same room, press `Cmd+Shift+5` on the Mac, choose *Record Entire Screen* and *Options → Microphone*. Online: a Zoom or Meet call where Mitul shares his screen and records; everyone uses their full name as display name.
- **Before recording:** Do Not Disturb on, fonts 18 pt or larger, run `make demo` and `make test` once, then `clear`. Open the 13 windows in the order listed in `docs/VIDEO_SCRIPT.md`, starting with `docs/video/title_card.png`.

Shot list

#	Time	Speaker	On screen
1	0:00-0:25	Mitul, then everyone	Title card with all five names
2	0:25-1:10	Akshat	Block diagram (Harvard half) → `program_counter.v`
3	1:10-1:55	Ajeesh	README program → `demo.hex` → `data_memory.v`
4	1:55-2:35	Vaibhav	Block diagram (shared half) → `address_decoder.v` → `simple_controller.v` line 28
5	2:35-3:40	Hardik	Terminal `make demo` → `waveforms.png`
6	3:40-3:58	Hardik	Terminal `make test`
7	3:58-4:35	Mitul	`no_data_access` line → report → GitHub page → end card



Scene 1 title card (`docs/video/title_card.png`)

11. The video: exact words

SCENE 1 | 0:00-0:25 | MITUL, THEN EVERYONE

On screen: title card.

Mitul: "We are Team HEXARCH from Section A. Our project is Project 9, Instruction and Data Memory Hierarchy."
Akshat: "I'm Akshat Chauhan." **Ajeesh:** "I'm Ajeesh Amreet." **Vaibhav:** "I'm Vaibhav." **Hardik:** "I'm Hardik Maheshwari." **Mitul:** "And I'm Mitul Bhatia."

SCENE 2 | 0:25-1:10 | AKSHAT

On screen: `architecture.png`, Harvard half.

"Our idea is simple. Instructions say what to do; data is the value being worked on. In a Harvard design they have separate memories and separate ports, so the controller can fetch the next instruction while the current one reads or writes data. In a von Neumann-style design, both live in one memory with one port, so they take turns. **[program counter]** This is our program counter. On each clock edge it adds one, but only when a fetch is accepted; otherwise it holds."

Driver: at **program counter**, open `rtl/program_counter.v`, point at lines 7-9.

SCENE 3 | 1:10-1:55 | AJEESH

On screen: README, the three-line program.

"Our program has three instructions: LOAD 0, STORE 1, HALT. **[In memory]** In memory they are the bytes 10, 21 and F0. The first hex digit is the operation; the second is the data address. Data address 0 starts at 7. **[data memory]** This is our data memory, sixteen bytes. A read is ready within the same cycle; a write happens on the rising clock edge. LOAD copies 7 into the accumulator, STORE writes it to address 1, and HALT stops."

Driver: at **In memory**, open `programs/demo.hex`; at **data memory**, open `rtl/data_memory.v`, lines 13-15.

SCENE 4 | 1:55-2:35 | VAIBHAV

On screen: `architecture.png`, shared half.

"Both designs hold thirty-two bytes in total. In the shared design, one memory has one port. **[address decoder]** Our address decoder puts instructions at addresses 0 to 15 and data at 16 to 31. **[This line]** This line in the controller is the key rule: in shared mode, fetch is switched off during a LOAD or STORE. So the program counter holds, and the next instruction is fetched one cycle later."

Driver: at **address decoder**, open `rtl/address_decoder.v` (lines 8-9); at **This line**, open `rtl/simple_controller.v` and highlight line 28.

11. The video: exact words (continued)

SCENE 5 | 2:35-3:40 | HARDIK

On screen: terminal. Driver types `make demo` and presses Enter.

"I'll run make demo. Each row is one clock cycle. F, R and W mean instruction fetch, data read and data write. In cycle 2, Harvard fetches STORE while LOAD reads data, so fetch and read are both 1. Shared fetch is 0, because its one port is busy. In cycle 3, Harvard fetches HALT while STORE writes the 7. Harvard executes HALT in cycle 4, so it finishes in four cycles. Shared needs six; the two extra cycles are the two blocked fetches. Both end with 7 at data address 1. **[waveform]** The waveform shows the same: Harvard fetch overlaps the read and the write; shared fetch never does."

Driver: point at rows 2, 3, 4 and 6 as they are mentioned; at **waveform**, open `docs/figures/waveforms.png`.

SCENE 6 | 3:40-3:58 | HARDIK

On screen: terminal. Driver types `clear; make test` and presses Enter.

"Our testbench checks all of this automatically. make test runs every scenario, including boundary addresses, an invalid opcode and a mid-program reset. All of them pass."

SCENE 7 | 3:58-4:35 | MITUL

On screen: the `make test` output; driver points at the `no_data_access` line (Harvard=4, shared=4).

"With no data accesses, both designs take four cycles. So separate paths help only when a fetch and a data access would compete; in our model the shared design needs exactly one extra cycle per LOAD or STORE. This is a small simulation model, not a general speed-up claim. **[report]** Our IEEE-format report **[repository]** and our repository have the Verilog, the testbenches, block diagrams, waveforms and results."

Then **one** of these, whichever is true: "Each of us also added our own test or improvement, which you can see in our commit history. Thank you." *or* "Next, each of us is adding our own improvement from our own GitHub account. Thank you."

Driver: at **report**, open the report PDF; at **repository**, open the GitHub page and scroll once; after "Thank you", show the end card for two seconds and stop.

If something goes wrong

- `make demo` fails: type `cat results/demo_output.txt` and say "This is the saved output of the same command."
- `make test` fails: type `clear; cat results/test_output.txt` and say the same sentence.
- A stumble: keep going. Restart only if two things go wrong or a name is missed.
- Past 4:40 during scene 7: skip straight to "Thank you".

After recording

□ Watch it once: all five names heard, code readable, at most 5:00.

□ Upload: YouTube **Unlisted**, or Google Drive with *Anyone with the link → Viewer*.

□ Open the link in a logged-out private window; it must play.

□ Put the link and duration in `VIDEO.md`, commit, push, then submit the form.